GitHub Copilot in the CLI

Choose the surface that fits the work

Session M2-01 | 3 hours

Agenda

Segment	Time
CLI fit and setup	10 min
Modes and checked plan critique	10 min
Programmatic use	10 min
Agents and fleet supervision	10 min
Controls, queue, and recovery	10 min
Surface choice and handoff	10 min

Start with the task

Choose the CLI when the work starts or ends in the terminal:

- reading logs or command output;
- working over SSH or in a container;
- running a repeatable command sequence;
- producing output for another script.

Use the IDE when visual navigation and diff review matter more.

Match the mode to the risk

Mode	Use it when
Standard	The task is small or still changing
Plan	You want to review the approach before edits
Autopilot	The task has clear limits and a recovery plan

Permissions still apply. More autonomy needs a narrower task.

Revise the plan before accepting it

```
/plan Find the three failing behaviors. Name the files and tests for each fix.
Do not edit project files.
```

Add a constraint: preserve the response fields and status codes.
Ask which steps change. Check the diff before approval.

Save both plans and the assumption you challenged.

Ask a different critic before approval

```
/rubber-duck Critique the repair plan against the source and tests.
Look for a hidden defect or response-contract change. Do not edit files.
```

Rubber duck is built in and read-only. Copilot selects a contrasting model when one is available. Check the findings yourself.

Inspect `/subagents` Proactive invocation and leave it unchanged. The opt-in persists and adds usage.

Fleet needs independent work

Plan separate read-only questions about routes, utilities, and test gaps.

1. Leave Plan mode and run `/fleet` with that breakdown.
2. Open `/tasks` and read the child-task results.
3. Check claims against files or focused tests.
4. Reconcile disagreements and inspect `/usage`.

Extra agents can use more credits.

Shared commands, different surfaces

Workflow	Surface
`/plan`, `/fleet`, `/review`	CLI and app
`/orchestrate`	App built-in skill; Session M7-05
`copilot -p` and shell pipelines	CLI
Cloud changes through `/delegate`	Session M3-02

State the limits of an automated task

- the working directory;
- the files or commands in scope;
- the expected result;
- the checks to run;
- when Copilot must stop.

Programmatic mode needs a contract

```
copilot -p "Summarize the failing test output"
```

Before another script uses the answer, define:

- stable input;
- a predictable output format;
- a failure signal;
- limited permissions.

`--output-format=json` emits **JSONL events**. Parsing each line doesn't show that the task succeeded or that the answer fits your schema.

CLI and IDE can share one workflow

```
CLI
  reproduce the failure
  capture focused test output
        ↓
IDE
  inspect the code and diff
  review the proposed change
        ↓
CLI
  rerun checks
  record the result
```

Switch surfaces when the work changes.

Inspect approval, sandbox, and limit separately

```
/permissions show
/sandbox status
/sandbox policy node --version
/limits
```

Approval decides whether a tool may run. The sandbox limits process access. Inspect both without weakening them.

AI-credit caps are soft. GitHub's references differ on reset scope, so check what your installed UI shows.

Queue, stash, and background are different

Control	Result
`Ctrl+Q`	Queue a follow-up while busy
`Ctrl+S`	Stash or restore an unsent draft
`Ctrl+X, b`	Move a running command to the background
`/tasks`	Show its state, output, and exit status

Use the lab's harmless 15-second command. Compare the queued summary with the real result. Background work stays in the CLI session and doesn't isolate files.

Recover before branching an alternative

1. Commit the checked repair.
2. Let Copilot create only a disposable note.
3. Open `/rewind` and restore conversation **and files**.
4. Check the note is gone and the repair remains.

Rewind doesn't undo external actions or replace Git rollback.

Context fork is not file isolation

Command	Result
`/fork`	Conversation fork, same files
`/fork worktree`	Context plus a new checkout of `HEAD`
`/worktree`	New checkout; uncommitted edits stay behind
`/move`	Move the uncommitted draft into a new worktree
`/resume`	Return to the parent; check its path, branch, and status

Move only the analysis note. Don't merge the alternative.

Save the work for review

Keep:

- the final diff;
- test output and exit status;
- generated reports;
- the prompts you used.

Another engineer should be able to check the work without the conversation.

Lab handoff

Use `/plan`, challenge an assumption, and approve the revision before the repair. Read `/fleet` results through `/tasks` and check the claims you accept. Check the critique against the source. Note the controls, the queue and background result, and the recovery and worktree state.

120 minutes: setup 15; repair 40; programmatic and fleet 35; recovery and choice 30.

Submit: passing tests, a reviewed diff, the health report, and `workflow-record.md` with the plan revision and usage.